



2D Shark Adventure

Ein Android-Handy-Spiel in Unity — entwickelt in C#. Ein Hai schwimmt vorwärts, frisst Fische und weicht Hindernissen aus. Steuerung mit einer Hand über Touchscreen.

PROJEKT

UNITY 2D

ANDROID

C#



Idee & Inspiration

Vorbilder

Inspiriert von **Flappy Bird** und **Hungry Shark** — einfach, süchtig machend, mobil.

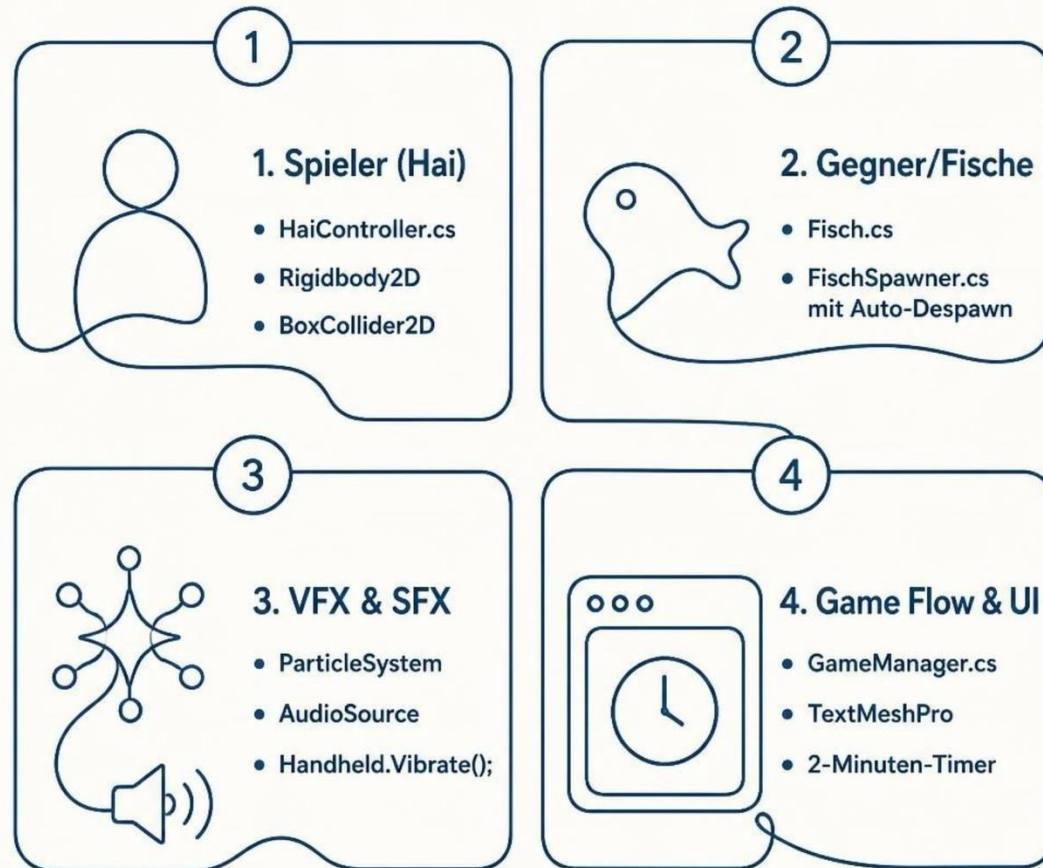
Spielprinzip

Hai schwimmt automatisch vorwärts. Touch-Button für Auftrieb. Fische fressen, Objekten ausweichen.

Ziel

Einhändige Bedienung auf dem Smartphone — intuitiv, schnell, unterwegs spielbar.

Technik: Kernkomponenten



Technischer Überblick

Das Spiel basiert auf vier zentralen Systemen: Spieler-Steuerung per Rigidbody2D, ein Spawner-System für Fische mit automatischem Cleanup, Partikel- und Audio-Feedback sowie ein GameManager mit Timer und UI-Steuerung.

Entwicklung: Phase 1–3

1

Phase 1: Setup

Unity auf Android umgestellt. Hai mit Rigidbody2D und Collider. **HaiController.cs** für Vorwärtsbewegung und Touch-Auftrieb.

2

Phase 2: Spawner

Fisch-Prefab mit Trigger-Collider. **FischSpawner.cs** instanziiert Fische in Intervallen. Auto-Despawn bei 15 Einheiten hinter Kamera.

3

Phase 3: Feedback

Partikelsystem mit Burst-Emission und Auto-Destruction. Haptisches Feedback via **Handheld.Vibrate()** auf Android.

Entwicklung: Phase 4–5

Endlos-Hintergrund & Audio

Zwei nahtlose Unterwasser-Sprites (*Underwater1* & *Underwater2*) werden durch `HintergrundWiederholung.cs` automatisch vor die Kamera teleportiert. Hintergrundmusik: „Sharks“ von Imagine Dragons als `AudioSource` auf der Main Camera.

Rundenzeit & Menü-Rückkehr

UI mit `TextMeshPro` um Timer und Countdown erweitert. `GameManager.cs` zählt 120 Sekunden runter. Bei Ablauf: „**ZEIT ABGELAUFEN!**“ → Eingabe stoppen → nach 2,5 s (`Invoke`) Szene `StartMenu` laden.

Entwicklung: Phase 6 — Visuelles Upgrade

Asset-Pack

„**Animated Low Poly Animals Pack**“ von Backrock Studios importiert — endlich kein Platzhalter mehr.

Neue Prefabs

Hai-Prefab und Fisch-Prefabs erstellt. Animationen gemäß Assets eingebaut — Hai und Fische bewegen sich jetzt lebendig.

Ergebnis

Das Spiel machte „*schon viel mehr Laune*“ — der visuelle Sprung war enorm.



Entwicklung: Phase 7–8 — Highscore & Enemy Shark

Highscore im Menü

Highscore wird persistent über **PlayerPrefs** gespeichert und im Startmenü angezeigt:

```
int alterHighscore =  
PlayerPrefs.GetInt("Highscore", 0);
```

Enemy Shark

Ein gegnerischer Hai wurde hinzugefügt, der nach einiger Zeit in die **entgegengesetzte Richtung** schwimmt — für mehr Unvorhersehbarkeit und Spannung im Spiel.

 Kollision mit dem Enemy Shark löst „Game Over“ aus.

Phase 9: Android-Build & Demo

01

Build Settings

Szenenreihenfolge festgelegt: **0: StartMenu**,
1: Game

02

Player Settings

Package Name, Minimum API Level,
Architekturen: **ARMv7 + ARM64**

03

APK & Test

APK gebaut, auf Android-Gerät übertragen und
live getestet

Test: Probleme & Lösungen

⚠ Audio-Import (.m4a)

Problem: Unity erkannte `.m4a` nur als Binärdatei ohne Audio-Optionen.

Lösung: Datei in `.wav` konvertiert.

⚠ APK-Parsing-Fehler

Problem: Android verweigerte Installation — „Beim Parsen des Pakets ist ein Problem aufgetreten“.

Lösung: Minimum API Level auf Android 8.0/9.0 heruntergesetzt, beide Architekturen (ARM64 & ARMv7) aktiviert.

Reflexion

Was gut lief

Iteratives Vorgehen in 9 Phasen
funktionierte gut. Spielbare Version früh
erreicht, dann schrittweise verfeinert.

Technisches Learning

Auto-Despawn-System verhindert
Speicherlecks. **PlayerPrefs** für
persistente Daten ist einfach und effektiv.

Ausblick

Enemy Shark zeigt: Mehr
Unvorhersehbarkeit macht das Spiel
spannender. Nächste Schritte könnten
Level-Design oder Power-Ups sein.